Many Hands Engineering

Notes on building the thing that builds the thing.

Matthew Sullivan

2026

About the author

I'm a senior software engineer with twelve years building platforms. I was a founding engineer on a Fortune 50 retailer's enterprise generative AI platform, where my work centered on agent orchestration and multi-agent systems. Before that, my background spans network automation, distributed-systems R&D (including a multi-backend distributed database with a custom cluster-state protocol), large-scale observability infrastructure, and developer-platform engineering at startups and agencies.

This handbook is the product of working through what changes when most of the code stops being typed by hand.

A note on the name and the claim

This handbook describes a framework I call **Many Hands Engineering**. The name is mine. The ideas are not — they come from distributed systems, complex adaptive systems, commons governance, swarm intelligence, cybernetics, and military command doctrine, among others. Where I've coined a term or renamed an existing one, I say so and name the lineage. The goal is a shared language teams can actually use, not a claim of novelty. Provenance is tracked openly in Part IV.

This is also not a definitive guide. It is **what I see as the direction software engineering is moving** — and the framework I've found most useful for working in that direction. Some of what follows is well-evidenced. Some is a working hypothesis informed by current practice. Some is an organizing metaphor that helps me think. I try to flag the difference where it matters. (See *What kind of claim is this?* at the end of Part I.)

A few places where this framework's vocabulary diverges from the field:

- I use **planned** and **emergent** as the main vocabulary where the open-source literature uses *cathedral* and *bazaar* (Raymond, 1997). Same distinction, plainer words.
- I use **signal** where the stigmergy literature uses *pheromone* or *trace*. Same mechanism, broader frame.
- I use **boundary** as an umbrella for the designed edges between regions — API contracts, review gates, capability scopes, trust zones. The literature splits these; in practice they are the same design move.
- I use **terrain** for the larger environment many hands operate in. Per-agent execution environments are increasingly being called *harnesses* (Hashimoto, OpenAI, Fowler, et al., 2026). Harnesses are a critical layer of terrain — see §4.2 for the relationship.
- I coined **placement** and **steward** because the concepts are real and the existing words don't quite carry them.

One name, one vocabulary, one mental model. That's the point.

How to read this

Four parts, each useful on its own.

Part I — The argument.
What's changing, why old questions are no longer enough, and the new one that is.

Part II — The framework.
The vocabulary, the spectrum, the placement discipline, the pace-layered platform, the commons.

Part III — The practice.
The anatomy of a loop, a catalog of real loops, the staged transition, the steward's shape, the economics of approval, trust and authority, measurement, correctness and security, where the framework applies and where it doesn't, and where to start on Monday.

Part IV — Foundations.
Lineage, mathematics, evidence, open questions. For the reader who wants to trace a claim back to its source.

One line, if that is all you take with you:

> *It used to be building the thing. Now it's building the thing that makes the thing.*

PART I

The argument

The turn

It used to be building the thing. Now it's building the thing that makes the thing.

Software is increasingly made by many hands. Some human, some automated. Some fast, some slow. Some narrow, some carrying judgment. None of this makes the old engineering wisdom obsolete — it makes it incomplete. We still need to know how to write good code. We also need to know what the old tools didn't quite answer: *what should be planned, what can be allowed to emerge, what should stay reversible, and where judgment should remain human.* That is what this framework is built around.

The unit of work has shifted before:

Era	Unit of work	Engineer's role	What mattered
Lines	The instruction	Author	Writing it well
Modules	The component	Architect	Arranging them well
Loops	The autonomous cycle	Steward	Shaping the conditions under which work happens

What the steward tends is the **terrain**: the substrate, signals, boundaries, and conditions under which autonomous work succeeds or fails. Good terrain compounds — each piece of work runs cleaner, safer, and faster than the last, and the team's attention is freed for the decisions that most need it. Bad terrain collapses — many hands work furiously and produce nothing that lasts.

The ideas underneath are older than the technology that makes them urgent. Termites build without blueprints. Common-pool resources are governed without tyranny. Armies delegate under uncertainty. Buildings learn. Cities zone. Distributed systems prove theorems about what they can and cannot promise. None of this is decoration — these are the substrates the framework sits on. The problems are not new; only the speed and scale at which we now face them.

Why "Many Hands"

> *Many hands can build a lot. They can also make a mess. The difference is the system around them.*

That's the whole picture. A shop with many skilled hands and no system produces noise. A shop with one skilled hand and a rigid system produces only what that hand can imagine. The craft is in the system — the conditions that let many hands work in parallel without stepping on each other, without losing coherence, and without losing the judgment that keeps the work honest.

"Many hands" includes human engineers, AI agents, review systems, automated tests, linters, deployment pipelines, on-call rotations, and every other source of action inside a modern codebase. It has always been many hands. The difference now is that some of those hands can make real decisions on their own — and that changes what the system around them has to do.

Why approval matters

The visible act of engineering in this era is the approval — the *looks good to me* on a pull request, the ship-it in a review, the considered yes at the moment where autonomous work meets human judgment. It can look trivial. It is not. When the system around the hands is built well, that approval is the load-bearing moment: the place where the work becomes real. A framework that doesn't take this moment seriously isn't a framework for software in this era. The mechanics of how approval scales — how it's earned, how it's budgeted, how it decays — are central enough that they get their own section in the practice (§3.6).

What kind of claim is this?

A framework only earns its keep if it's honest about what it knows. Three layers run through this document, and the reader should treat them differently:

- **Evidence-backed observations.** Things established by empirical research or hard-won industrial experience. Conway's Law, the impossibility theorems of distributed systems, Ostrom's commons design principles, the convergent failure patterns documented by major postmortems, the human-factors literature on automation. These are load-bearing.
- **Working design principles.** Patterns that follow from the observations and seem to hold across the cases I've studied or worked on, but haven't yet been validated at industry scale for the specific shape of agent-heavy engineering. Placement, the staged transition, the economics of approval, the five properties of trust. I commit to these because they're the best framing I've found, not because the science is settled.
- **Organizing metaphors.** Language that helps think clearly without claiming to be literally true. Terrain. The colony's immune system. The four dynamics. These earn their place by clarifying. They are not theorems.

Where it matters, I flag which layer a claim sits at. Where I don't flag, I'm usually in the second layer — describing what I think is the right move given what we currently know. The reader who treats every line of this document as gospel is missing the point. The reader who treats every line as opinion is also missing the point. The interesting work is in knowing which is which.

This handbook will be wrong in specific ways. It should be revised when evidence demands. The structural claims — that the spectrum is real, that positive feedback is dangerous, that commons need governance, that variety must match variety, that humans move to lower-frequency higher-authority roles, that trust must be earned, narrow, conditional, revocable, and expiring — are well-supported. The specific operational recommendations are bets, informed by theory and early practice. **The framework is a principled hypothesis, not a proven methodology.** Use it accordingly.

PART II

The framework

2.1 — Vocabulary

A small vocabulary, used consistently. Each term is paired with the closest industry or academic analog, so readers can cross-reference without losing the thread.

The two modes — the spectrum of how work is coordinated.

- **Planned work.** Specified in advance, decomposed into bounded tasks, executed against the spec, verified at scheduled checkpoints. Industry: design-by-contract, formal methods, waterfall, milestone-driven delivery. Lineage: Raymond's *cathedral* (1997).
- **Emergent work.** Expressed as constraints and goals, coordinated through signals in a shared environment, integrated continuously, verified through convergence. Industry: trunk-based development, reconciliation loops, bazaar-style open source. Lineage: Raymond's *bazaar* (1997); stigmergy (Grassé, 1959).

Most real systems are mixes. The design work is deciding which mode belongs where.

The six terms this framework introduces or reclaims.

Term	Definition	Closest industry analog
Loop	A closed cycle of work: *signal → action → verification → commit or revert*. The operating unit.	Reconciliation loop, control loop, autonomous workflow
Terrain	The designed environment loops run in: substrate, signals, boundaries, and shared resources. Literal, not metaphor.	Developer platform, substrate, the *environment* in an agent SDK
Signal	A decaying piece of metadata in the terrain that biases the next action. Test status, staleness marker, telemetry, confidence score, ownership tag. The artifact is not the signal; the signal is what the artifact *tells the next actor*.	Pheromone (stigmergy); state in reconciliation systems
Boundary	A designed edge between regions of the system. Contracts, review gates, capability scopes, trust zones. Where planned and emergent meet, a boundary holds the seam.	API contract, bounded context, trust boundary, review gate
Placement	The discipline of deciding, explicitly, where a decision lives on the planned/emergent spectrum. *The placement is the decision; the implementation follows from it.*	ADR-level design — but sharper and earlier
Steward	The human role: shapes terrain, sets intent, codifies norms, approves what crosses boundaries, holds veto on the irreversible.	Platform engineer, tech lead, architect — each partial; the steward is none of these alone

Two more that appear throughout.

- **The commons.** The shared substrates through which emergent work coordinates — the repository, the vector store, the eval harness, the deployment pipeline, the production namespace. Commons are *governable resources*, not passive storage. They degrade without governance. Lineage: Ostrom (1990).
- **Pace layers.** The strata of a system, each with its own characteristic rate of change — governance (years), architecture (months), capabilities (weeks), orchestration (days), agents (hours), invocations (minutes). Each layer changes at its own pace. Mixing signals across layers is a category error. Lineage: Stewart Brand (1994).

The four forces that shape every terrain.

- **Reinforcement** — a signal strengthens with use.
- **Decay** — a signal weakens with time.
- **Runaway** — reinforcement without counter-force. The death spiral.
- **Drift** — slow divergence between what the system measures and what the world rewards.

Most of what goes right or wrong in a many-hands system shows up as one of these four. Underneath them is **entropy** — the second-law tendency of any system to drift from order toward noise unless work is continually invested. Decay, drift, and (eventually) runaway are how entropy presents itself in software. The work against entropy is one of the steward's main jobs (§2.11).

2.2 — The planned / emergent spectrum

The planned/emergent spectrum is not a rhetorical device. Every property along it varies together:

	Planned ←	**→ Emergent**
Authority	Central	Distributed
Knowledge	Concentrated	Dispersed
Spec role	Prescriptive	Descriptive
Integration	Scheduled, synchronous	Continuous, asynchronous
Verification	Against the spec	Through convergence
Coupling	Explicit, audited	Implicit, tolerated
Coordination	Direct command	Environmental signal
Failure	Hard — the whole plan breaks	Soft — local decay
Strength	Guarantees	Adaptivity
Weakness	Brittleness	Commons degradation

These properties cluster because they reinforce each other. A decision made in a central, spec-verified way will also be integrated on a schedule and fail hard when wrong. A decision made in a distributed, convergence-verified way will integrate continuously and fail softly. Mixing halfway — central authority with continuous async integration, for example — usually produces governance theatre: the form of planned work without its substance, and the name of emergent work without its benefits.

At a boundary, guarantees and adaptivity trade against each other. You can't have both in the same region. You can have them in different regions, connected by well-designed boundaries.

2.3 — Placement, the central discipline

> *In a world of many hands, placement is the craft.*

Traditional architectural arguments are framed as *what*. What framework, what language, what pattern, what data model. This framework reframes them as *where*. Given that we need authentication, where on the spectrum does it belong? Planned — irreversible, regulated, coupled to everything. Given that we need internal developer tooling, where? Emergent — the cost of a wrong choice is a revert, and the variety of needs exceeds what central design can anticipate.

That reframe changes what evidence matters. An argument about *what* often ends in taste and authority. An argument about *where* ends in a check of three axes, which are empirical. You don't need to agree on what authentication *ought* to be to agree that it is irreversible, specified, and tightly coupled — and therefore planned.

The three questions that place any decision. For any decision of consequence:

- **Reversibility.** If this goes wrong, how expensive is it to undo? Cheap → emergent. Expensive → planned. *The most important question.*
- **Legibility.** Is the problem well-specified and stable, or exploratory and shifting? Specified → planned. Exploratory → emergent.
- **Coupling.** Does a local change force changes elsewhere? Tight → planned. Loose → emergent.

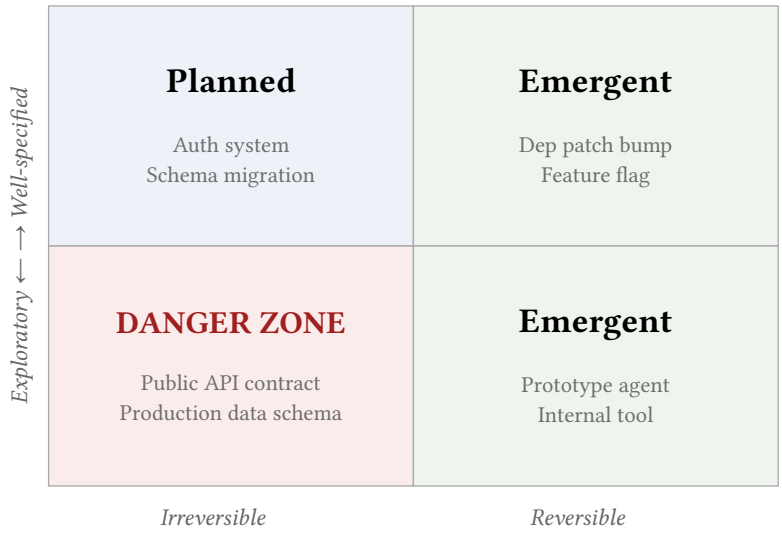

Figure 1: Where does this decision belong?

The bottom-left — *irreversible **and** exploratory* — is where engineering disasters live. Never let agents forage there without first making the decision reversible or making the problem legible.

Tight coupling pulls a decision toward the planned pole. Regulatory, financial, or safety surface pulls it further that way.

Do this explicitly for every decision of consequence. Write the placement down. **The placement is the decision.** The implementation follows from it. The approval at the end is honest only when the placement upstream was done with care.

2.4 — Five patterns for composing planned and emergent

Most real systems are hybrids. Five patterns recur.

1. **Shell and core.** A firm planned interface around an emergent interior. Microservice contracts, OS kernels, constitutions.
2. **Scaffold and release.** Planned builds the initial structure; emergent takes over. *Failure mode: overstaying the planned phase.*
3. **Nested zones.** Planned zones contain emergent zones contain planned zones. Handle things at the lowest layer that can handle them.
4. **Alternating breath.** The two modes take turns in time: divergent, convergent, divergent, convergent. Know which phase you're in.
5. **Zoning.** Risk-stratified coexistence. Payments are planned; feature flags are emergent. Same system, different zones.

2.5 — The pace-layered platform

A real many-hands platform isn't one system but several, stacked by rate of change.

Rate of change	Layer	Examples
Years	Governance	Policy, values, accountability, compliance posture
Months	Architecture	Platform primitives, schemas, contracts, trust model
Weeks	Capabilities	Tools, connectors, retrieval, eval harnesses
Days	Orchestration	Agent graphs, routing, delegation
Hours	Agents	System prompts, strategies, tool selections
Minutes	Invocations	A single task run

Three rules govern the stack.

- **Each layer changes at its own pace.** Governance that changes weekly is theatre. Invocations that need architectural review are a slow IDE.
- **Signals flow up; authority flows down.** A pattern observed at the invocation layer can bubble up into a capability, then architecture, then governance — but only by ratification at each layer, not by fiat from below. Conversely, a governance change reshapes the constraints every layer below operates under.
- **Each layer's failure mode is characteristic.** Governance fails by becoming irrelevant. Architecture fails by becoming brittle. Capabilities fail by proliferating without curation. Orchestration drowns in combinatorial complexity. Agents hallucinate and drift. Invocations throw transient errors. Pitch reviews at the layer of the actual failure.

The planned/emergent distinction applies *within* each layer. The pace layer tells you *when* something changes; the spectrum tells you *how* it's coordinated.

2.6 — The commons, governed

Every shared substrate in a many-hands platform is a commons — a resource many actors draw from and deposit into, where over-use degrades the resource for all. Elinor Ostrom's decades of field research on common-pool resources produced eight principles that long-enduring commons share. They translate to many-hands platforms almost line for line.

1. **Clear boundaries.** Every emitter has an attributable identity; every consumer a stated interest.
2. **Congruence.** Rules fit the resource. High-frequency signals decay faster; high-consequence signals need stronger evidence.
3. **Collective choice.** Those affected by a rule can modify it.
4. **Monitoring.** Watchers are accountable to users, not to a distant authority.
5. **Graduated sanctions.** A first offense is met lighter than a repeated one. Earned autonomy, narrowing envelopes after incidents, approval rates that rise and fall with track record.
6. **Low-cost conflict resolution.** If resolving conflict is expensive, conflict will be hidden — and hidden conflict is worse than open.
7. **Rights to organize.** Teams and sub-platforms have meaningful local autonomy. Overriding every local decision destroys the local knowledge that made the decision good.
8. **Nested enterprises.** Large commons are layered commons, each with its own rules at its own scope.

Ignore these and the platform eventually exhibits the tragedy of the commons — not from villainy, but from absence of governance. Embrace them and the platform becomes durable the way long-lived open-source projects and healthy markets are durable. *Strong evidence: Ostrom's work is among the most empirically grounded bodies in the social sciences. The mapping to software commons is more recent but the mechanism is the same.*

2.7 — The goal, restated

The goal isn't to build the product. **It's to build the conditions under which the product gets produced.** The engineer's attention moves upstream — to the design of the terrain, the tuning of signals, the setting of norms, the placement of decisions — while the downstream work runs well enough not to demand that attention back.

That reframes nearly every traditional metric. Velocity isn't the point; *ratio of autonomous to supervised throughput* is. Test coverage isn't the point; *reversibility of the average change* is. Lines of code isn't really a signal at all.

2.8 — The dynamics

A many-hands platform is a dynamical system in the formal sense. It has states, transitions, stable and unstable equilibria, and phase transitions as parameters cross critical thresholds. Treating it as static architecture is a category error the system will eventually punish. You don't have to do the math, but you should know what it says.

Signals spread, fade, and accumulate — a dynamic called reaction-diffusion. Three rates determine everything: how fast a signal spreads, how fast it fades, and how strongly it's reinforced. Decay too slow and signals saturate into noise. Reinforcement outruns decay and the system locks onto whatever was reinforced first, even if it was wrong. Mathematically: without sufficient decay, self-reinforcing aggregation can grow without bound. That's the shape of many runaways you'll eventually see.

Positive feedback produces phase transitions. Self-reinforcing dynamics change *qualitatively* when a control parameter crosses a critical value. Below, perturbations die. Above, they grow without bound unless something counter-acts. Discontinuity is the default; adding one more agent can change everything.

Exploration and exploitation trade against each other. Every system coordinating through signals has a knob — explicit or hidden — for how much agents follow existing signal versus wander. Too exploitative and the system locks on the first good-enough path. Too exploratory and nothing consolidates. Mistuning this knob is the most common cause of either brittleness or noise.

Coordination cost is topological. Direct messaging among N agents requires N^2 channels. Shared-environment coordination requires N. That's why colonies scale and tightly-coupled microservice meshes degrade above a few tens of services. Combinatorial, not ideological.

Some things are impossible. CAP, PACELC, FLP — theorems, not preferences. You can't have consistency, availability, and partition-tolerance at the same time. You can't have deterministic async consensus tolerating even one failure. No architecture evades these. You choose where to pay the cost.

A regulator must match the variety it regulates. Ashby's Law. A one-size-fits-all safety filter can't regulate a variety-rich agent fleet. Guardrails must be at least as differentiated as the failures they guard. Not negotiable.

Intelligence turns coordination into political economy. When agents can model incentives, they can deceive, free-ride, and game evaluators. The system's stable outcomes tend to be the ones the incentive structure supports — whether or not you intended them. *Designing the incentives is, increasingly, part of designing the system.*

2.9 — The two death spirals

Many-hands systems fail in two characteristic families. Learn to recognize each from the first symptom.

The emergent death spiral

A reinforcing signal without counter-force runs away. The biological archetype is the *ant mill* — army ants who lose the main trail follow each other in a circle until the colony dies of exhaustion. The signal did what signals do; the system lacked the mechanism to break the loop.

In software, the same dynamic takes many forms. Retry storms overwhelm the service being retried. Agents cite each other until ungrounded claims look well-grounded. Recommendation loops narrow onto local optima the exploration rate can no longer escape. Evaluators trained on agent output drift into rewarding surface features that agents then optimize for directly.

	Healthy emergent work	Runaway
Decay	present	absent
Variety	preserved	collapsed

	Healthy emergent work	Runaway
Outside reference	intact	severed
Net behavior	self-correcting	self-amplifying

Warning signs: signal-to-noise declining while activity rises; the same patterns recurring across unrelated tasks; high confidence on outputs a human recognizes as degraded; eval scores diverging from real outcomes; variety collapsing without a corresponding increase in selectivity.

Remedies, always in this family: inject variety; increase decay; add an *outside reference* the loop cannot produce itself; tighten the trust model; in severe cases, evacuate and restart. **The problem is in the terrain, not in any individual agent.** Patching the agent won't fix it.

The planned death spiral

A specification loses contact with reality faster than it can be updated. Denver airport baggage, NHS NPfIT, the original Healthcare.gov — all the same pattern. Requirements changed during development; the spec couldn't keep up; implementation proceeded against a spec that was increasingly wrong; integration at the end revealed the gap, catastrophically.

In many-hands systems, it looks structurally identical. A central orchestrator plans fully before executing in a domain where full planning is impossible. A top-level coordinator becomes a bottleneck for every decision. A spec-first workflow locks in an interface that turns out wrong. A single reviewer becomes the bus-factor-of-one.

Warning signs: the backlog of exceptions grows faster than the plan can absorb them; WIP accumulates because no piece integrates until the whole is ready; a few humans become permanent bottlenecks; outputs are superficially correct but fail only in composition; the specification becomes ceremony.

Remedies: shorten the integration cycle; decentralize what doesn't need central authority; turn the spec from document into executable contract (fewer words, more tests); distribute approval by domain.

Hybrid failures

The most dangerous failures are hybrids — they have the form of one mode and the substance of the other. A heavily-specified system whose specs are never enforced is planned in form, emergent in substance, and inherits the weaknesses of both. A lightly-specified system whose informal norms have ossified into uneditable custom is the opposite. Governance theatre, ivory-tower architecture, the inner-platform effect: all hybrids. *The honest test is not what mode the system claims, but what mode its actual decisions are made in.*

2.10 — Guardrails, the system's immune response

Guardrails aren't add-ons. They're load-bearing design work. They fall into four classes, each answering a different question.

- **Structural — can this even happen?** Enforced by the medium, not by policy. Schema validation at write time. Capability scoping. Sandboxing. Type systems. Cheapest and most reliable — forbidden actions are simply impossible.
- **Reactive — did this just happen, and should it continue?** Rate limits, circuit breakers, backpressure, content filters, test gates. Run at the moment of action. They must not depend on the

thing they're guarding — a circuit breaker that relies on the service it's breaking against isn't a circuit breaker.
- **Dynamic — is the system drifting?** Approval rates per agent and class, error-budget burn rates, eval drift, canary analysis. Operate over time. Catch patterns single-event guardrails miss. The main design challenge is avoiding false-positive fatigue.
- **Reflective — are our guardrails still working?** Postmortems that reveal missed failure classes. Chaos engineering probing the guardrail layer. Meta-evals that check whether the eval harness is still representative. The most important class, because without it the other three decay silently.

A mature platform has guardrails at all four layers and recognizes that the four answer different questions.

Grade each guardrail explicitly along five axes:

Axis	Question	Red flag
Coverage	What fraction of the failure class does it catch?	Less than you thought
Latency	Time from trigger to firing	Slower than the harm rate
Blast radius	Damage done by the time it fires	Your real cost
False-positive rate	Fraction of spurious firings	Trains operators to ignore
Recovery cost	Work to return to a good state	Can exceed the failure cost

Grading tends to reveal that many guardrails are doing less than they appear to. A review gate with a 95% approval rate is rubber-stamping. A rate limit set at ten times peak is decorative. The exercise forces the question of whether each guardrail earns its keep.

Guardrails are themselves subject to the dynamics they guard. They can run away (a retry policy causing the overload it was protecting against). They can Goodhart (an eval used as a gate will be optimized against). They can atrophy. They can become theatre. The defenses: add decay (rotate evals, retire stale rules), add variety (multiple independent guardrails for critical failures), add reflection (measure guardrail effectiveness), add escape valves (a guardrail that can't be overridden will be overridden destructively). Fractal, as the pace-layered view predicted.

2.11 — The work against entropy

Every terrain decays. This is not pessimism — it is thermodynamics, applied to software. Documentation goes stale. Tests rot. Dead code accumulates. Evals drift away from the work they were meant to measure. Conventions ossify into dogma. Trust grants outlive the conditions they were earned under. Branches abandoned in the corner of the repository quietly turn into landmines.

Entropy is the default. **Order requires work.** A many-hands platform fights entropy on three fronts at once.

The first front is decay — the explicit, designed kind. Signals that don't fade become noise. Trust grants that don't expire become liabilities. Conventions that don't get questioned become cargo cults. The framework has already named decay as one of the four dynamics (§2.1) and built it into multiple loops (the dependency-patch loop's track record is recent, not lifetime; the freshness signal in the doc loop resets and re-decays). *Designed decay is how the terrain stays current.*

The second front is garbage collection — loops whose only job is to fight accumulation. The harness engineering literature (§4.2) documents this directly: agents that run periodically to find inconsistencies in documentation, violations of architectural constraints, dead code that has crossed a threshold of disuse. They are unglamorous and load-bearing. Without them, the commons fills with cruft, and at some point the cruft is what the next agent reads first.

The third front is adversarial pressure — the deliberate disturbance of an otherwise functioning system to keep it honest. This is the part teams skip because it feels counterintuitive: things are working, why break them?

Why deliberate disturbance is necessary

Systems that aren't disturbed converge on whatever path got reinforced first. The exploration knob (§2.8) decays alongside everything else, and the system locks onto a local optimum that may have been wrong from the beginning. Many-hands platforms are particularly vulnerable to this because feedback compounds: an agent that succeeds gets more work, which generates more success signals, which earn more autonomy — until the conditions change and nothing in the loop notices. Without periodic disturbance, the platform becomes increasingly confident in increasingly stale conclusions.

The remedy is not stability — stability without disturbance is brittleness in disguise. The remedy is *productive instability*: small, deliberate disturbances that prevent ossification, force re-adaptation, and keep the terrain honest about what it actually depends on.

Three forms of adversarial pressure are worth naming:

- **Chaos engineering.** Netflix's Simian Army didn't just test for resilience to failure — it ensured the system never came to depend on the absence of failure. In a many-hands platform, the analog is broader: deliberately failing tools, simulating model regressions, randomly revoking permissions, killing background loops, corrupting non-critical signals.
- **Adversarial verification.** Property-based testing, mutation testing, fuzzing, the specification-by-adversary loop (§3.3). Instead of asking *does this work?*, ask *what would it take to break this?* — then build the agent that systematically tries.
- **Forced removal.** Periodically take something away — a tool the agents rely on, a piece of context they always read, a signal they always check — and see what happens. If they collapse, you've found a hidden dependency. If they adapt, the dependency wasn't load-bearing. *Subtraction reveals what addition concealed.*

The discipline, in practice

Adversarial pressure isn't a one-time exercise. It's a recurring practice with its own pace layer.

- **Daily**: small disturbances built into normal operation — random retry jitter, cache misses, occasional eval shuffling.
- **Weekly**: deliberate fault injection in non-critical paths, mutation tests on recent changes.
- **Monthly**: chaos game days, full red-team passes against new agent classes, reviews of trust grants for evidence of staleness.
- **Quarterly or on triggers**: removal experiments — take away a context source, a tool, a heuristic, and observe what breaks.

This isn't testing. Testing asks whether the system meets a known specification. Adversarial pressure asks whether the system is *quietly relying on things no one specified.* The first is verification. The second is discovery. Both matter; they are not the same.

The healthy posture is *equilibrium under disturbance* — a system that holds its shape not because nothing is challenging it, but because it has built the capacity to absorb challenge and re-form. That is what an

immune system actually does. That is what guardrails (§2.10) aspire to. **Entropy fought too well becomes its own pathology** — the system that has been over-stabilized loses the capacity to adapt to what comes next. The work of fighting entropy, done right, is not the elimination of disorder but the maintenance of *responsive* order.

2.12 — The shape of a healthy system in motion

Learn the signature. It's worth more than any checklist.

- **Variety is high but bounded.** Different agents solve similar problems differently, within a range the team recognizes. Outliers exist and are a sign of health; their absence signals premature convergence.
- **Flow is steady.** Arrival and completion rates match over moderate windows. Sudden WIP accumulation signals a bottleneck; sudden evaporation signals silent drops.
- **Escalations are rare but real.** When humans are called, it's for something that genuinely needs judgment. Frequent escalations ⟶ the autonomy envelope is too narrow. Zero ⟶ too wide, or the system has stopped surfacing real problems.
- **Signals correlate with outcomes.** When evals say quality is up, users say quality is up. *When internal signals diverge from external reality, it's the earliest and most important warning that the system has begun to optimize for its own metrics.*
- **The commons is clean.** Stale signals pruned. Abandoned branches die. Unused tools retired. A commons slowly filling with cruft is slowly becoming uninhabitable.

When all five hold, the system is, in the framework's sense, **alive**. Keep it alive — not by freezing it, which kills it, but by maintaining the conditions under which it stays a healthy dynamical system.

PART III

The practice

3.1 — A running example

Throughout this section I use a single concrete example: the **dependency-patch loop**. It's small, reversible, present in nearly every codebase, and ordinary enough to show what the framework actually looks like in motion.

> *A signal watches the dependency graph for available patch-level upgrades with clean changelogs. When one appears, an agent opens a change with the bump and the quoted changelog. CI runs. On green, the change merges; on red, it stays open for human review, and the agent's track record on this class updates. Signals produced: patch cadence, CI pass rate, post-merge rollback rate — all of which feed back into how aggressively the next patch is applied, and whether this agent has earned autonomy on this class.*

It's a loop. It has placement (reversible, legible, loose coupling → emergent). It has a boundary (CI as a gate; the human reviewer as a higher gate). It has a steward (the person setting the thresholds). It has trust that's earned and revocable. We'll come back to it throughout.

3.2 — The anatomy of a loop

Every loop — the smallest and the largest — has the same anatomy: five stages that close on themselves.

1. **Signal.** A trace in the terrain fires the loop.
2. **Action.** An agent does scoped work.
3. **Verification.** An outside reference checks the work.
4. **Commit or revert.** The result either lands or rolls back.
5. **Update.** The outcome feeds back into the terrain as new signal — decaying over time, biasing the next loop.

The steward holds veto on the irreversible at stage 4.

A loop is healthy when all five stages exist, are bounded in time and scope, and close — meaning the outcome (pass or fail) feeds back into the terrain as a signal the next loop can read. Loops that don't close aren't loops; they're fire-and-forget scripts that accumulate debt.

Every loop needs six ingredients. A loop missing any of them isn't a loop yet — it's a script looking for a home.

Ingredient	What it is	What happens without it
Signal	A trace in the terrain that fires the loop	Loop never starts, or fires on wrong conditions

Ingredient	What it is	What happens without it
Bounded action	Scoped to a specific class of change	Scope creep; blast radius exceeds reasoning
Verification	An outside reference: tests, eval, human judgment	The loop self-approves; drift; Goodhart
Reversibility	A cheap, well-understood undo path	Mistakes ossify; stewards stop trusting the loop
Signal update	Outcome feeds back into the terrain	The next loop can't learn from this one
Authority surface	Explicit rules for when autonomy expands or contracts	Trust becomes a single global decision; fragility follows

The dependency-patch loop has all six. Signal: *patch available, clean changelog*. Bounded action: *bump version, run tests, open PR*. Verification: *CI suite*. Reversibility: *git revert*. Signal update: *CI result and post-merge rollback feed the agent's track record*. Authority surface: *auto-merge eligibility by task class and track record*.

3.3 — A catalog of real loops

These are skeletons — intentionally independent of any specific orchestrator, model, vendor, or pipeline. Each is a pattern that recurs across mature software platforms. The fidelity is in the shape, not the technology.

A note on specifications, before the catalog. Specs are a tool used by several of the loops below, not the framework's philosophy. They earn their cost at **boundaries** — contracts with external systems, interfaces between planned and emergent regions, the schemas agents emit. They waste their cost in deep emergent territory, where exploration updates the direction and rigid specs foreclose exactly that. Treat specs like software: version-controlled, tested, amended by learning. The specification-by-adversary loop below shows how to convert a spec from document into continuously-stressed process — powerful at boundaries, weaker where intent resists specification.

Starter loops — high reversibility, narrow scope, clean signals

Begin here. Each has a clear trigger, a clear verification, and cheap reversal. The goal of the first loop isn't the loop; it's the template.

Dependency-patch loop — described above. The canonical starter.

Stale-doc refresh loop. A signal on each doc page records *last code change in covered area* and *last doc update*. When the gap exceeds a threshold, an agent rewrites the page against current code, flagging sentences where it couldn't confirm a claim. The change ships with a reviewer tagged from the code-ownership file. *Signal produced:* a freshness field that decays with code change and resets on review.

Flaky-test loop. CI records pass/fail per test per run. A signal surfaces when pass rate drops into a flaky band. An agent attempts one of three scoped interventions — seed isolation, timing fix, or retry-with-diagnostic — documented in the change. If flakiness persists after the attempt, the test is quarantined with an owner assigned. *Signal produced:* a flakiness score per test that compounds across runs.

Dead-code loop. Coverage, import graph, and call telemetry combine into a *deadness* signal. Code that's been dead across a stability window gets a deprecation proposal; after another window with no usage, an agent opens a deletion change. Reversal is cheap — the change is a single revert. *Signal produced:* a persistent map of cold regions that informs refactoring.

Middle loops — wider scope, richer signals, still reversible

Once several starter loops run cleanly, richer loops become possible because the terrain now carries signal worth reading.

Triage loop. A planner agent reads the full signal field — incident telemetry, fragility map, freshness scores, backlog — and orders the next work. It doesn't do the work itself; it produces a ranked queue with rationale. *Signal produced:* an audit of which ranking criteria correlated with real business outcomes, which tunes the ranker.

On-call assist loop. When an alert fires, an agent pulls the runbook, recent deploy history, relevant dashboards, and the last three similar incidents. It drafts a hypothesis and the three most diagnostic next checks. The human on-call reads, corrects, and acts. *Signal produced:* alignment between agent hypothesis and actual root cause, which becomes a confidence signal for increasing autonomy on narrow incident classes.

Refactor-candidate loop. A signal combines churn, coupling, fragility, and ownership clarity into a refactor-pressure score. When a cluster exceeds threshold, an agent proposes a refactor plan — not the refactor itself — with costs, risks, and a small prototype. A human decides whether to authorize execution. *Signal produced:* a history of where refactor pressure predicted real architectural debt.

Test-backfill loop. A signal identifies code with high churn and low coverage. An agent writes tests that pin current behavior, marking each as *behavioral* (locking current behavior) or *specified* (testing against documented intent). Behavioral tests are flagged for review because they encode whatever the code currently does, correct or not. *Signal produced:* a coverage map weighted by criticality.

Security-patch loop. A signal listens for CVE advisories and matches against the dependency graph. An agent opens a change per affected package, with the advisory, affected path, and suggested version. For non-breaking upgrades in non-critical paths, auto-merge is eligible once the agent has earned the approval on this class; for anything else, a human approves. The loop explicitly distinguishes *availability of a patch* from *application of a patch. Signal produced:* mean time from advisory to remediation, per criticality tier.

Advanced loops — require mature terrain

Attempt only after starter and middle loops are durable.

Feature prototype loop. A product intent — expressed as a small set of acceptance assertions and UX sketches — becomes a sandbox where one or more agents build competing implementations against the assertions. A human compares results on feel and fit and chooses a direction. The loop doesn't replace product judgment; it parallelizes the *preparatory* work so judgment has more options. *Signal produced:* a library of rejected implementations, which becomes a rich training signal about taste.

Specification-by-adversary loop. For a well-bounded interface, the team writes intent as assertions. Adversarial subagents attempt to falsify each assertion against the implementation. An assertion that survives sustained attack is treated as verified; one that falls triggers a human decision — is the implementation wrong, or was the assertion wrong? Most valuable at boundaries; degrades in deep emergent territory where intent resists specification. *Signal produced:* an executable, continuously-stressed specification that outlives its authors.

Migration loop. A long-running migration — database schema, framework version, API deprecation — is decomposed into a sequence of independently-reversible steps. Each step is a loop: prepare, migrate a shard, verify, then either commit and move to the next shard or roll back and pause. The steward owns the commit-to-next-step decision; the agent owns the shard work. *Signal produced:* migration progress as a visible, resumable state, which ends the pattern of migrations that stall invisibly at 40%.

Capacity-routing loop. A meta-agent observes approval rates per task class and routes incoming work to the agent whose track record best matches the task. Agents that fail get less work; agents with a stronger track record get more within their class. The steward sets the routing policy and the gate thresholds. *Signal produced:* a per-agent specialization pattern the team can read, reason about, and adjust.

Disturbance loop. On a schedule, a deliberately-disruptive agent introduces controlled stress into the terrain — fails a tool that's normally available, removes a piece of context other agents have come to rely on, simulates a model regression, revokes a permission temporarily, corrupts a non-critical signal. The point isn't to find bugs; the point is to make sure the system doesn't quietly come to depend on the absence of disturbance. Other loops are observed for graceful re-adaptation versus collapse. Collapse surfaces a hidden dependency for the steward to address; graceful re-adaptation builds confidence that the dependency wasn't load-bearing in the first place. *Signal produced:* a map of fragile assumptions and resilient ones, which informs where the next round of hardening belongs. (See §2.11 for the discipline this implements.)

How loops compose

Individual loops are useful. *Loops that read each other's signals* are where compounding lives.

The dead-code loop produces a map of cold regions. The refactor-candidate loop reads that map as one of its inputs. The test-backfill loop avoids cold regions and prioritizes hot-but-fragile ones. The migration loop reads coverage from the test-backfill loop to decide which shards are safe to move first. Each loop is simple; the *field of signals* they collectively maintain is rich.

No loop calls another. Each reads the terrain and writes to it. The coordination is in the environment, not in any orchestrator.

3.4 — The staged transition

Teams that jump straight from traditional engineering to autonomous agents almost always stumble. The stumble is rarely in the agents. It's in the terrain they were dropped into.

The transition that works proceeds in stages, each making the next possible.

Stage	Move	What it creates
0. Legible terrain	Instrument the environment for non-human readers and writers before any agent does meaningful work	Structured metadata, signal schemas, standardized tool interfaces
1. Close one loop	Pick one small, reversible loop and make it work end-to-end	A working template

Stage	Move	What it creates
2. Replicate	Build more loops of the same shape. Resist broadening any one	Platform capacity
3. Coordinate	Let loops interact through signals in the terrain	Cross-loop coordination and guardrails
4. Plan	Introduce planners and routers that read the full signal field	Higher-order agents
5. Earn the approval	Class by class, move specific agent-task pairs from *propose* to *auto-apply with sampling review.* An agent earns the thumbs-up on a narrow class by building a track record (see §3.6)	Per-agent, per-class autonomy
6. De-automate gracefully	Maintain the ability to revoke an earned approval when a failure mode outgrows it	Durable reversibility

Not a strict sequence — stages overlap, different regions can be at different stages. But the order is real. You can't skip Stage 0. You can't run Stage 4 before Stage 3. You can't responsibly run Stage 5 without Stages 2 and 3 being durable.

The stage most often skipped is Stage 0. Skipping it produces smart agents acting on dumb terrain: high activity, low signal, no compounding. *The harness engineering literature (§4.2) is the deepest current account of how to do Stage 0 well — architectural constraints encoded as linters and structural tests, knowledge stored in the repository rather than in conversation, "garbage collection" loops that fight entropy. If your team is at Stage 0, that body of work is essential reading.*

The most seductive mistake is broadening Stage 1 before replicating it. A dozen simple loops compounding beats one sophisticated loop burning attention.

The dependency-patch loop walks through this staging naturally. Stage 0: the repo needs to expose dependency metadata, changelogs, and CI status as structured signals. Stage 1: one agent, one loop, human review on every PR. Stage 2: three or four similar loops around docs, flaky tests, dead code. Stage 3: the loops start reading each other's signals — patch decisions consider flakiness history. Stage 4: a triage agent decides which kinds of patches to tackle first. Stage 5: the patch agent earns auto-merge on clean patch-level bumps with a track record above threshold. Stage 6: when the model underneath changes, or the repo restructures, the earned autonomy is revoked until it's re-earned.

3.5 — The steward's shape

The single most persistent misconception about many-hands systems is that they reduce human involvement. They don't. They **reallocate** it — from the high-volume, low-stakes middle to both ends: upstream to the design of the conditions, and downstream to the adjudication of cases that exceed autonomy.

- **Upstream: intent, taste, and the terrain itself.** Agents don't generate purpose; they pursue it. Someone decides what "good" means, where acceptable variance falls, what the system is actually *for*. This work can't be delegated because it's the work that defines what delegation means.
- **At pace-layer boundaries: codifying norms.** When an agent practice consolidates into a convention — a useful pattern imitated across agents — the steward notices, ratifies, and promotes it from emergent practice to explicit rule. *This is how emergent work ratchets forward instead of drifting nowhere.*
- **At boundaries: curating contracts.** Every interface between planned and emergent is a living contract. Maintaining it requires understanding both sides, anticipating change in both, and making revisions that minimize breakage. Invisible when done well, catastrophic when done poorly.
- **Downstream: adjudication.** The autonomy envelope is deliberately set so some cases escalate. When they do, the steward's judgment defines how similar cases will be handled going forward.
- **Periodically: deliberate disturbance.** The steward is responsible for keeping the system honest about what it depends on. That means scheduling adversarial pressure — chaos days, removal experiments, red-team passes — even when nothing seems wrong. *Especially* when nothing seems wrong. A system that hasn't been disturbed in months has accumulated dependencies the steward doesn't yet know about. (See §2.11.)
- **Throughout: veto on the irreversible.** Production deploys. Data deletions. External calls with cost. Legal commitments. The platform proposes; only a human approves. This isn't a position on AI's limits — it's a position on the cost of being wrong.

Shape: lower-frequency, higher-authority than the traditional engineering role. Fewer actions, each with more leverage. If engineers in a mature platform find themselves on more execution and less decision, the ratios are inverted.

The atomic unit of the steward's authority is the considered thumbs-up. Not rubber-stamped, not reflexive — the deliberate approval at the moments that matter, backed by the upstream work that made the moment considerable. A steward who approves everything isn't stewarding. A steward who approves nothing hasn't built the terrain to trust.

What goes wrong with the human in the role

The steward's role is harder than it looks, and the failures are well-documented in fields that automated decades before software did. The aviation and medical literature converged on the same observations, and they apply here.

- **Automation bias.** When a system is mostly right, humans stop checking. They accept what the system says even when something else in front of them disagrees. Stewards who review at high volume and high approval rate drift into this without noticing.
- **Skill atrophy on rare failures.** The cases that need a human are by construction rare and unusual — exactly the cases the human now has the least practice on. A steward who hasn't thought through a hard case in months is not as sharp on it as one who used to do that work daily.
- **Monitoring fatigue.** Watching a system that mostly works is harder than working. Attention drifts. Anomalies that would jump out to a fresh observer slide past a saturated one.
- **Substitution illusion.** Adding automation often doesn't reduce human work — it changes its shape, and the new shape is sometimes harder. The steward role isn't a smaller version of the engineer role; it's a different role with its own cognitive demands.

The defense isn't willpower. It's terrain. *Triangulation* (multiple metrics, sampled deep review, adversarial probing) catches what a tired steward misses. *Rotation* prevents any one person from carrying the role too long. *Practice cases* — deliberately working through hard examples even when none arrive — keep

skills warm. *Time upstream* — explicitly budgeted time for terrain design, not just approval — is what makes the role sustainable. The steward's job is to build a system where the human in it can succeed.

3.6 — The economics of approval — gate, rate, budget

The steward's approval is scarce by design. *Looks Good To Me* is engineering shorthand for that approval — the considered yes a human gives at the moment where autonomous work meets judgment. Three operational terms describe how that scarcity is managed; together they form the throttle between agent output and human judgment.

- **The gate.** The boundary where a human confirms an agent's proposal. Gates are placed per task class. A proposal that crosses a gate requires a considered approval before it commits. Proposals that don't cross a gate auto-apply within their autonomy envelope.
- **The rate.** The fraction of gated proposals that earn approval, tracked per agent, per task class. An agent's track record is the signal that justifies moving the gate (Stage 5).
- **The budget.** The steward's finite capacity for considered approvals per unit time. Not a wish — a real constraint, like an SRE error budget.

The three are coupled. You can't change one without changing the others.

Lever	What it controls	Failure if mistuned
Gate	Which proposals need human approval vs auto-apply	Too wide ⟶ budget exhausted; too narrow ⟶ commons degrades
Rate	Fraction of gated proposals approved	Drifts up ⟶ rubber-stamp; drifts down ⟶ agent misaligned
Budget	Considered approvals per steward per unit time	Too low ⟶ bottleneck; too high ⟶ no capacity for upstream work

The governing inequality, in plain language:

> *Proposals entering gates per week ≤ approval budget × (1 – false-approval tolerance)*

When inflow exceeds the budget, a queue builds. Queues produce one of two pathologies: stewards rubber-stamp (approval rate drifts up artificially) or the system stalls (WIP accumulates). Both are signals, not solutions.

The healthy adjustment is always structural. If the budget is saturated, the fix isn't heroics — it's either (a) widen the autonomy envelope for classes where the approval rate is consistently high, moving them below the gate, or (b) narrow the envelope where the rate has drifted, moving them above. The lever that stays constant is the budget. Stewards aren't supposed to work harder; they're supposed to place the gates better.

The three together describe what "earning the approval" means operationally. An agent earns it by producing proposals at a rate compatible with the budget, with an approval rate high enough that moving the gate is justified. Moving a gate isn't a decision made on feel — it's what happens when the numbers say the gate belongs somewhere else.

A concrete example. A team of four engineers shares a steward role on a rotation. Each rotation gives the on-call steward a budget of roughly 30 considered approvals per week — enough to review carefully without burning the rest of their attention. The dependency-patch loop generates 18 proposals a week with a 96% approval rate over the last quarter. Of the 30-approval budget, this loop currently consumes 18. That's most of the budget on a single class. The team moves the gate: patch-level bumps with green CI auto-merge; only minor-version bumps and any failing CI come to the steward. The loop now costs 3 approvals a week instead of 18. The reclaimed budget covers the new on-call assist loop the team just stood up — or, on a slow week, frees the steward to spend that attention upstream on terrain design. The numbers told the team where the gate belonged. The steward made the move.

3.7 — Authority, trust, and the cost of being wrong

> *Useful is not the same as trusted. Readable is not the same as authoritative.*

A system can let an agent see something without letting it act on it. A system can let an agent act in one region without letting it act in another. A system can trust an agent's work on Tuesday and re-verify it on Wednesday when a dependency changed overnight. In a many-hands platform, **authority is a design dimension, not an afterthought.**

Guardrails (§2.10) cover the *mechanics* of containment. This section covers the *philosophy* of trust — what it is, how it's earned, and how it comes apart.

Five properties of real trust

Earned autonomy in a many-hands system has five properties. Any missing one is a place where the system will eventually be surprised.

- **Earned.** Trust accumulates through track record on a specific class of work, not by fiat or vendor claim.
- **Narrow.** Trust on one class doesn't transfer to another. An agent that has earned auto-merge on patch bumps has earned nothing on schema changes.
- **Conditional.** Trust depends on the environment it was earned in — the model, the tools, the permission set, the codebase shape. The environment is part of the trust.
- **Revocable.** Trust can be withdrawn instantly on evidence of failure. The cost of keeping a bad grant is always higher than the friction of revocation.
- **Expiring.** Trust decays over time if not renewed by fresh track record. An agent that hasn't acted in a class for months starts from a lower baseline.

These five together define what it means to **earn the approval** — and they're what distinguishes an autonomy grant from a blank check.

The practical rule

> *Trust belongs to a task class in a specific environment. Change the environment, and some trust resets.*

The dependency-patch example shows what this looks like in motion. Suppose the agent has a clean track record on patch bumps — forty-seven merged, zero rolled back. Then one of three things changes:

- **The model underneath changes.** The pipeline upgrades to a new base model, or a new orchestration layer. The environment the trust was earned in is no longer the one producing the proposals. *Trust drops from auto-merge back to propose-plus-review until the new model re-earns it.*
- **The permission set changes.** The agent gains access to new parts of the repo, or a new external API. *The new surface isn't covered by the old track record.* Auto-merge persists on the old class; the new surface starts at proposal-only.
- **The repository itself changes.** A major refactor, a new framework version, a migration that reshapes what a "patch bump" even means. *The old track record is no longer predictive.* Autonomy resets for a calibration window until fresh track record exists.

None of these require detecting a failure. The trust resets because the environment it was anchored to has shifted.

Information vs authority

A pattern worth making explicit: **the right to read is not the right to write, and the right to write is not the right to deploy.** In a many-hands system, each of these is a separate grant, evaluated separately.

- **Read authority** — what an agent can see. Default: wide, so agents have context. Narrow only where secrets, personal data, or competitive information require it.
- **Write authority** — what an agent can propose. Default: narrow, with explicit class-by-class expansion.
- **Commit authority** — what an agent can merge without human approval. Default: none, earned per class through track record.
- **Deploy authority** — what an agent can cause to run in production. Default: never autonomous on the irreversible. Agents propose; humans approve.
- **External authority** — what an agent can do to the world beyond the repo (API calls with cost, emails, filings, trades). Treated as deploy authority: never autonomous without explicit, class-level human grant.

This isn't bureaucracy. It's recognition that the blast radius of a wrong action depends on which of these is engaged, and that confusing them — treating read authority as if it implied write authority, for example — is how many of the worst failures happen.

What counts as irreversible

The reversibility axis (§2.3) is the single most important placement question, and it relies on a shared team judgment about what "irreversible" actually means. A useful working definition:

> *An action is irreversible if undoing it costs more than not doing it.*

That definition is deliberately practical. It includes the obvious cases — data deletion, external financial transactions, legal commitments, production traffic changes that affect customers — and it includes the less obvious ones where the *information* released can't be recalled (a leaked secret, a public announcement, an email sent). The team should maintain an explicit list of what counts as irreversible in its context, and review it when the system changes.

The steward's veto applies to the irreversible. Everything else can, in principle, be delegated along the graduated trust progression.

How trust comes apart

Trust in a many-hands system decays in predictable ways. Watching for these is part of the steward's job.

- **Silent environmental drift.** The model, tools, or permissions changed without anyone resetting the autonomy envelopes that assumed the old environment.
- **Class boundary erosion.** Agents start doing work *adjacent* to the class they were trusted on. The track record doesn't transfer but the auto-merge grant wasn't tightened.
- **Invisible rubber-stamping.** The approval rate stayed high, but the stewards were skimming rather than reviewing. The number no longer means what it used to.
- **Reputation drag.** An agent that does well on easy instances of a class gets auto-merge; the hard instances get rubber-stamped because the easy ones trained the steward to trust.

Each is countered by the same move: triangulate the trust signal. Track record alone can lie; track record plus sampled deep review plus adversarial probing plus post-merge defect rate cannot all lie at once. The deliberate disturbance practice in §2.11 is the systematic version of this — adversarial pressure applied to trust grants, on a schedule, before the trust has had a chance to silently expire.

3.8 — What to measure, and what to leave for judgment

Every metric is a proxy, and every proxy can be gamed. Neither measure nothing (flying blind) nor measure everything (Goodhart buffet). Measure a small number of things at each pace layer, triangulate to defeat single-axis gaming, and reserve the important questions for judgment.

Pace layer	Good metrics	Notes
Invocation	Latency, cost, error rate, rollback rate, tool-call failure, escalation rate	Operational: is it working, how expensively?
Orchestration / Agent	Eval scores, **eval drift**, trajectory variety, *ratio of autonomous to assisted*	The last is the headline metric of the transition
Capability	Coverage, integration time, **obsolescence rate**	High inflow + low retirement = accumulating cruft
Architecture / Governance	DORA metrics, SPACE-style orthogonal metrics	Lagging and coarse *by design*

Reserve for judgment, not metric: the quality of abstractions, the coherence of the system's direction, the taste of the output, the health of the norms. These can't be measured; attempts produce either superficial proxies (lines of code, sentiment scores) or sophisticated proxies that eventually get gamed. Assign them to humans. Trust the judgment.

The triangulation rule. If a metric will be used as a target, add at least two independent metrics that would be harmed by gaming it. Velocity alone will be gamed. Velocity + change failure rate + rollback rate cannot be gamed without visible cost in the others. **Triangulation is the single most practical defense against Goodhart in a platform.**

3.9 — Correctness, behavior, and security

The framework so far has been mostly about coordination, maintainability, and trust. Those are necessary. They are not sufficient. A platform whose loops compound and whose commons stays clean can still ship code that does the wrong thing or opens a hole an attacker can walk through. *Authority answers who is allowed to do what; correctness and security answer whether what got done was right.*

The honest gap, surfaced by Birgitta Böckeler in her early read of the OpenAI harness work, applies to this framework too: it's easier to specify how code is organized than whether it does what users actually need. Architectural rigor and functional correctness are different problems. The first yields to mechanical enforcement. The second requires deeper feedback between the system and the world it claims to serve.

Three commitments belong in any serious many-hands platform.

- **Behavior verification, not just artifact verification.** Tests that pin current behavior aren't the same as tests that confirm intended behavior. The terrain needs evals, contracts, or property-based checks that bind the system to *what it should do*, not just *what it currently does*. Where the difference matters, the difference must be visible — behavioral tests marked as such, distinct from specification-bound tests (the test-backfill loop in §3.3 does this explicitly).
- **Security as a distinct dimension of trust.** The five properties of trust in §3.7 apply to functional correctness. They also apply, separately, to security posture. An agent earned auto-merge on a class of changes — but did its track record cover injection-prone surfaces? Did it cover authorization-sensitive ones? Did the model that earned the trust have the same security training as the one running today? These are different questions from the functional ones, and they fail differently.
- **Adversarial pressure on outputs, not just on infrastructure.** §2.11′s disturbance practice operates on the terrain itself. The same posture applies to outputs: red-team the changes the system ships. Adversarial probes, fuzzing, mutation tests, security scans — at agent cadence, not at quarterly audit cadence. The attackers are using the same tools as the defenders. The defenders need their tools running continuously.

The framework gives these the weight they deserve by naming them explicitly: *correctness and security are first-class concerns, not downstream consequences of good coordination.* Where they are not addressed in detail in this handbook, the reader should treat that as a gap I'm acknowledging, not one I'm hiding. The literature on secure-by-design, adversarial ML, and verification-aware development is deep and the right place to go for the specifics. The framework's contribution is to insist these belong in the conversation, not at its edge.

3.10 — Boundary conditions

A framework that doesn't say where it doesn't apply is a framework that hasn't earned its claims. Many Hands Engineering is most useful in some conditions, less useful in others, and explicitly worse than the alternatives in a few.

Where Many Hands tends to compound:

- Greenfield or near-greenfield systems where the terrain can be designed deliberately from the start.
- Codebases with strong, fast tests that give cheap, reliable verification.
- Domains where most changes are reversible and most decisions can be made in the bazaar mode.
- Teams with the budget and discipline to invest in harnesses and terrain before scaling agents.
- Tasks that decompose cleanly into bounded loops with clear signals.

Where Many Hands struggles or doesn't apply well:

- Brownfield codebases full of accumulated entropy, where the cost of building a legible terrain may exceed the cost of doing the work the old way.
- Safety-critical or regulated domains (avionics, medical devices, financial settlement, nuclear) where the planned mode is correctly dominant, the verification regime is heavyweight by law, and emergent coordination is a category error rather than a tool.
- Highly ambiguous product work where the question isn't *how do we build this* but *what should we build* — a domain where human judgment carries most of the load and many-hands compounding offers little.
- Verification-hard problems where checking correctness is much harder than producing output (security-sensitive cryptography, novel ML architectures, complex distributed protocols). The framework still applies, but the gates stay tight and the budget stays small for a long time.

A specific honesty: the strength of single, contiguous agent work. Some of the most productive work I've seen comes from a skilled engineer pairing closely with a single capable agent for hours on a focused problem — no orchestration, no multi-agent coordination, no platform overhead. Just one expert and one good tool, in flow. Many Hands Engineering doesn't replace that mode. It addresses the *other* mode: when the work has scaled past what one expert and one agent can hold in a single session, when the team is shipping more than any individual reads, when the coordination across loops is the bottleneck. Both modes will coexist. A team that always works in the many-hands mode is over-engineering; a team that never does is leaving compounding on the table.

There is also a possible future I want to acknowledge cleanly: capable enough single agents may eventually subsume some of what this framework addresses. If a single sufficiently-capable agent can hold the entire context, do the placement, run the loops, and exercise the judgment, the multi-hand coordination layer becomes thinner. I don't think we're there. I think the framework is useful in the world we have, and probably in the world we'll have for the meaningful future. But I'd rather name the possibility than pretend it isn't on the horizon.

3.11 — Monday morning

When a team reads this and asks *what do we do first*, the answer is almost always the same: **pick one small closed loop and make it work end-to-end.** Not the most interesting loop. Not the highest-ROI loop. The one you can fully instrument, test, and control within a short cycle. Dependency patches are the canonical starter — clean signals, clean reactions, cheap reversal, and humans generally dislike doing them.

The point of the first loop isn't the loop. It's the template. Once one works end-to-end, the second is dramatically cheaper, the third cheaper still, and by the fifth the team has a pattern they apply almost mechanically. **The platform grows by adding loops, not by making any single loop smarter.**

Everything in this handbook — the spectrum, the pace layers, the commons, the guardrails, the trust model, the measurements — is in service of making the *nth* loop cheaper than the *(n-1)th*. A team with one working loop and confidence it can replicate the pattern is further along than a team with elaborate architecture and no working loop.

Start there.

PART IV

Foundations

Lineage, mathematics, evidence, open questions. Flagged honestly. This section is a reference, not a primary read.

4.1 — Intellectual inheritance

- **Biology and swarm intelligence.** Grassé's 1959 coinage of *stigmergie* for termite construction. Theraulaz and Bonabeau formalizing the mechanism. Dorigo's Ant Colony Optimization (1992⟶) extracting it into a rigorous algorithmic framework. *Strong evidence.*
- **Economics and distributed knowledge.** Hayek 1945 on prices as compressed signals of dispersed knowledge. **Ostrom's *Governing the Commons* (1990)** on how decentralized institutions succeed where markets and central planning fail. *Strong — one of the most empirically grounded bodies in social science.*
- **Open source.** Raymond's *The Cathedral and the Bazaar* (1997, 1999). The origin of the vocabulary this framework renames to planned/emergent. The change is mine because the new terms travel better across technical audiences and across the planned/emergent distinction's many applications beyond open source itself. *The conceptual debt is total.*
- **Organizations.** Conway 1968 — organizations ship their communication structure. Simon 1962 — near-decomposability. Brand 1994 — pace layering. *Strong across decades.*
- **Cybernetics.** Ashby 1956 — requisite variety as a lower bound on regulation. Beer — viable system model. *Strong theoretically; mixed in institutional application.*
- **Military command.** Prussian *Auftragstaktik*, modern mission command — specify intent, delegate means, trust the unit. *Strong historical evidence.*
- **Software and systems.** Brooks 1975 for planned-mode pathologies. The distributed-systems canon (CAP, CRDTs, Kubernetes reconciliation, gossip protocols) as the engineering vocabulary for emergent coordination. The formal-methods tradition (TLA+ at AWS and Azure, B method at RATP, property-based testing) as the engineering vocabulary for planned guarantees. *Strong both directions.*
- **Human factors.** Bainbridge 1983 — the ironies of automation. Research on automation bias, complacency, skill atrophy. *Strong across aviation, medicine, and now AI-assisted engineering.*
- **Security and authority.** Least-privilege design from Saltzer & Schroeder (1975). Capability-based security (Dennis and Van Horn, 1966; modern work at Cambridge, MIT). Zero-trust architecture (NIST SP 800-207, 2020). *Strong; the authority model in §3.7 is a direct adaptation.*
- **Philosophy of complexity.** Scott 1998 — *Seeing Like a State*, legibility vs *metis*. Alexander 1965 — "A City is Not a Tree." *Mixed-to-strong depending on the claim.*

4.2 — Harness engineering, a closely related discipline

A related body of practice converged in early 2026, under the name **harness engineering**. The chronology is worth naming explicitly because it shows the field arriving at similar conclusions independently and quickly:

- **February 5, 2026** — Mitchell Hashimoto publishes the originating definition: *"anytime you find an agent makes a mistake, you take the time to engineer a solution such that the agent never makes that mistake again."*
- **February 11** — OpenAI's *Harness Engineering: Leveraging Codex in an Agent-First World* documents an internal experiment shipping roughly a million lines of production code with no human-typed source.

- **February 17** — Birgitta Böckeler (Thoughtworks) publishes the first analysis memo identifying the convergence and naming the gaps (notably: maintainability is addressed; functional correctness is not).
- **March 2** — Stripe documents an evaluation harness for coding agents.
- **March 12** — HumanLayer publishes on coding-agent harness configuration.
- **March 24** — Anthropic publishes design notes for long-running application harnesses.
- **April 2** — Böckeler publishes a fuller framework, with categories of guides, sensors, and harness templates.

The core observation across these sources matches mine: as agents take on more of the writing, the engineer's primary work shifts to *designing the environment in which the agent operates.* OpenAI summarized it as *"designing environments, feedback loops, and control systems."* That sentence could be the spine of this framework.

The relationship to Many Hands Engineering is one of scope. Harness engineering focuses on the per-agent execution environment — tools, context architecture, structured documentation, architectural constraints, custom linters, garbage-collection loops. Many Hands Engineering sits one level up: how multiple harnessed agents share a commons, where decisions are placed on the planned/emergent spectrum, how authority is granted and revoked, how work compounds across loops, how human stewardship operates at a different rate than agent execution. *A harness is a critical layer of terrain.* Specifically, the per-agent layer. A team with excellent harnesses but no shared terrain produces reliable per-agent work that fails to compound. A team with broad terrain but weak harnesses has agents that can't be trusted to do anything.

The two are convergent, not competing. The harness engineering practices documented in early 2026 map directly onto Stage 0 of the staged transition (§3.4) — the substrate layer that everything else builds on. Where harness engineering is more developed, this framework defers to it. Where Many Hands extends — multi-agent coordination, the commons, the trust economy, the steward's role, placement — it builds on the harness layer rather than replacing it.

4.3 — Mathematical backbone

A note before the list: the results below are real mathematics. In this framework, they function primarily as **analogies and design intuitions**, not as predictive models a team can compute against. CAP and FLP genuinely constrain what distributed systems can promise. Mean-field games genuinely model populations interacting through aggregate effects. Keller-Segel genuinely produces blow-up without decay. But the bridge from the math to a specific operational decision in your platform is currently informal — the framework hasn't (and probably shouldn't) try to give you state variables or measurable thresholds to compute against. Treat these as the substrate of why the framework's instincts hold up, not as equations to solve.

- **Reaction–diffusion / Keller–Segel** — without sufficient decay, self-reinforcing aggregation produces finite-time blow-up. The mathematical form of runaway.
- **Bifurcation** — self-reinforcing dynamics change qualitatively at critical thresholds. The canonical form of trail formation in emergent systems.
- **Mean-field game theory (Lasry–Lions; Huang–Malhamé–Caines)** — many strategic agents interacting through aggregate effects; equilibrium as a consistency fixed-point.
- **ACO convergence (Gutjahr; Stützle–Dorigo)** — reinforcement systems converge to optimum *only* with a positive lower bound preserving exploration. The floor is not decorative.

- **CAP / PACELC / FLP** — consistency, availability, partition-tolerance aren't simultaneously achievable. Latency trades against consistency even without partitions. Deterministic async consensus tolerates no failures.
- **Ashby's Law** — regulator must match regulated in distinguishable states.
- **Coordination complexity** — full-mesh N^2; shared-medium N. Combinatorial, not ideological.

4.4 — Evidence base, weighted

- **Very strong:** the mathematical results; distributed-systems impossibility theorems; the empirical record of major failures (Knight Capital, AWS US-EAST-1, Meta BGP 2021, CrowdStrike 2024, XZ Utils, NPfIT, Healthcare.gov); automation-bias research; DORA/Accelerate practice–performance correlations; Conway's Law; Ostrom's design principles.
- **Strong:** the shape of effective engineering practices (CI, small batches, observability, progressive delivery); formal methods in safety-critical domains; the broad outline of where autonomous coding agents succeed (narrow, reversible, well-tested) and fail (ambiguous, cross-cutting, novel).
- **Mixed:** vendor productivity claims for AI coding assistants; benchmark numbers for autonomous agents; specific multi-agent orchestration patterns versus strong single-agent baselines at equivalent compute; the Spotify model; "edge of chaos" as a design principle.
- **Sparse:** long-term skill-atrophy effects; optimal human intervention frequencies; spec-driven development with AI at industrial scale; adversarial spec testing case studies; industry-wide failure telemetry for autonomous coding agents.

This is the distribution behind the *epistemic stance* declared in Part I. The structural claims fall in the very-strong-to-strong band. The specific operational recommendations — placement, the staged transition, approval economics, the five properties of trust — fall in the strong-to-mixed band. None of the framework rests on sparse evidence; where it touches sparse territory, I say so.

4.5 — Open questions

- What are the phase-transition thresholds for multi-agent systems? The mathematics says they exist; the specific numbers aren't established.
- What is the failure-mode distribution of autonomous coding agents in production at scale? Most interesting data is proprietary.
- How do teams effectively move agents from supervised to autonomous work? The staging in Part III is a principled proposal, not a validated one.
- What are the long-term effects of heavy agent reliance on engineering skill, organizational knowledge, and commons health? Historical parallels from aviation and medicine suggest significant effects; whether they manifest in software, at what cadence, is unknown.
- How should specifications and agents relate in mature workflows? The intuitive case is strong; the empirical case is young.
- How does earned trust actually transfer — or fail to transfer — across model upgrades? The theory says it shouldn't; the economics say teams will try anyway.

These aren't flaws in the framework — they're the frontier of the field. A team holding the framework lightly in these areas, treating it as a starting point for its own learning, will outperform one treating any

recommendation as settled. **The spectrum applies to the framework itself: its invariants are planned, its specific practices are emergent.**

Coda

Software is built by many hands. Some human, some not. Some fast, some slow, some carrying judgment.

The old engineering wisdom still matters. It's no longer enough.

What's needed now is a craft of *placement* — knowing what should be planned and what should be allowed to emerge, where reversibility must be preserved and where judgment must stay human. Loops are the operating unit; signals are how they coordinate; boundaries hold the seams; stewards hold the intent. Trust is earned, narrow, conditional, revocable, and expiring. Entropy is the default; deliberate disturbance is how the system stays honest about what it depends on. The approval at the end is the moment the work becomes real.

The framework will be wrong in specific ways, and should be revised when evidence demands. But the shape of it — many hands, a system around them, placement as the central discipline, humans upstream and downstream but not in the middle — is, in the view this handbook defends, durable.

Build accordingly. And when the terrain is built well, and the loops are running, and the work is worth approving — say so.

> *It used to be building the thing. Now it's building the thing that makes the thing.*